

Lab 05 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — 950 MB refused by security

Answer. (1) **The source code ships with the image:** anyone who pulls it can read the application — and often local configuration files too. (2) **The attack surface:** JDK, Maven, `git`, `curl`, a full shell, hundreds of Debian packages — each one a tool for an attacker who gains code execution, and together a 300-CVE scan report that nobody will work through. (3) **The `~/m2` repository** holds the downloaded artefacts and, frequently, a `settings.xml` with the credentials for the private Maven repository. Without multi-stage, (2) is the hardest to fix: you can `rm` the sources and `.m2` (poorly — the layers keep them, lab 02), but you cannot strip the JDK out of an image that starts from a JDK image.

Why. The final image is the last `FROM` plus whatever you add on top; there is no way to "subtract" the base image. Only a second `FROM` on a minimal base, combined with `COPY --from`, changes the base.

Nuance. Size does carry an operational cost (pull time during an incident, registry storage), but it is the weakest argument to bring to a security officer — what the image contains matters more than what it weighs.

Example.

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version; ls ~/m2 2>/dev/null'
```

Question 2 — Three `FROM`s

Answer. The **last** `FROM` produces the final image; the other two are temporary environments, destroyed when the build ends (only their cache survives). If no `COPY --from` (and no later `FROM`) references the second stage, Buildah **skips it entirely** — it is never built. The output shows it: the stages are numbered `[1/3]`, `[2/3]`, `[3/3]`, and the number of the unused stage never appears.

Why. The engine first derives the dependency graph between stages from the `--from` references, then builds only what leads to the target (the last stage, or the one named by `--target`).

Nuance. BuildKit does the same, and on top of that builds independent stages in parallel. An "unused" stage has a legitimate purpose: a test stage (`RUN mvn test`) that CI builds only with `--target test`.

Example.

```
podman build -f Dockerfile.unused -t u . 2>&1 | grep STEP      # [2/3] and [3/3] only, never [1/3]
```

Question 3 — 420 MB and the sources

Answer. `COPY --from=build /app /app` copies **the entire working directory** of the `build` stage: `Api.java`, `src`, `pom.xml`, `target/` with its classes, and the JAR. Fix: copy only the artefact.

```
COPY --from=build /app/target/api.jar /app/api.jar
```

(and adjust the `ENTRYPOINT` to `/app/api.jar`.)

Why. Multi-stage filters nothing on its own: the final image receives exactly what `COPY --from` asks for. Ask for a folder, and you get everything in it.

Nuance. Does the fixed image still contain a `.m2`? No — `.m2` sits in `/root` of the build stage, not in `/app`. But that is luck, not a guarantee. The rule: copy **named files**.

Example.

```
podman run --rm --entrypoint ls api-multi:1.0 /src      # No such file or directory: good sign
```

Question 4 — `ng serve` in production

Answer. Four reasons: (1) `ng serve` is a **development** server — unoptimised, no compression, no HTTP caching, and its own documentation says it is not meant for production; (2) the image contains **Node, the sources and `node_modules`** (often over 1 GB): a large attack surface and a code leak; (3) the build does not happen **once** but at every start, in watch mode, with *source maps* enabled; (4) the application and its tooling are not separated — a vulnerable development dependency runs in production. Instead: run `ng build --configuration production` in a Node stage, then `COPY --from` the `dist/.../browser` folder into an `nginx:alpine` image, with an nginx configuration that returns `index.html` for Angular routes.

Why. A compiled Angular front end is static. Serving it takes nothing more than a file server; everything else is build work, and build work belongs in the discarded stage.

Nuance. There is one case where Node stays in production: server-side rendering (Angular SSR / Universal). That is a **different** application with its own Dockerfile — and still not `ng serve`.

Example.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'  # 64 MB versus 167 MB (without node_modules!)
```

Question 5 — `UnsatisfiedLinkError` after Alpine

Answer. The PDF generation almost certainly depends on a **native library** (`.so`) compiled for `glibc` — fonts, rendering, compression. Alpine ships `musl`: the loader rejects the library, and Java throws `UnsatisfiedLinkError`. The command that would have exposed it: `ldd --version` in each image (`musl libc` versus `GLIBC`). The migration should have been tested against the **real workloads** (not just `/actuator/health`), on a staging environment, with a rollback plan — and decided component by component.

Why. A native library is bound to one specific `libc`; it is not portable bytecode. The two-week delay is no mystery: the PDF job may only run at month end.

Nuance. Alternatives exist: the `eclipse-temurin:21-jre-ubi9-minimal` image (Red Hat, `glibc`, ~100 MB), or `gcompat` on Alpine (fragile). Nor is this a Java problem only: Python and Node with native modules fall into exactly the same trap.

Example.

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --version 2>&1 | head -1' # musl libc
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version | head -1' # GLIBC 2.xx
```

Question 6 — Multi-stage and secrets

Answer. An `rm` creates a layer that hides the file, but the layer that wrote it stays in the image (lab 02). A discarded stage, by contrast, is **not** part of the final image: none of its layers is there. A secret written only in a discarded stage exists nowhere in the published artefact. Multi-stage stops protecting you when the secret gets **copied** into the final stage (`COPY --from=build /app` with the secret inside), when the artefact itself absorbed it (an `application.yml` with a password packed into the JAR), or when the secret passes through an `ARG` of the final stage (visible in `history`).

Why. Final image = layers of the last `FROM` + layers produced by its instructions. An earlier stage contributes only what a `COPY --from` extracts from it.

Nuance. The modern approach is `RUN --mount=type=secret`: the secret is available for one instruction, in any stage, and never becomes a layer. Multi-stage remains the structural guarantee; the *secret mount* is the per-instruction one.

Example.

```
podman build --secret id=pw,src=pw.txt -f Dockerfile.secret -t sec .
podman run --rm sec ls /run/secrets # No such file or directory
```

Question 7 — JAR as a block versus layers

Answer. (a) One 50 MB layer that changes at every build: the `push` and every `pull` move **50 MB**. (b) Four layers — dependencies (~45 MB, unchanged), loader (~1 MB, unchanged), snapshots (0), application (~5 MB) — so a deployment moves **~5 MB**. A tenfold difference. Yet (a) remains acceptable: 50 MB crosses a datacenter network in about a second, the JRE (180 MB) is shared either way, and (b) adds an extra stage, a different `ENTRYPOINT` (`org.springframework.boot.loader...` or `java -jar` on the folder), and complexity someone has to explain.

Why. Transfer happens layer by layer, differentially. What counts is the size of the layer that changed, not the size of the image.

Nuance. (b) starts paying off when you deploy frequently to many nodes, or over a slow network (edge, remote sites). The principle also works without Spring: splitting `lib/` (stable) from `classes/` (volatile) is enough.

Example.

```
podman history api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6 # one 50 MB layer, or four layers
```

Question 8 — 90 seconds turned into 7 minutes

Answer. The new agent starts with an **empty cache**: the build cache (layers) lives on the machine that builds, so a fresh agent — or an ephemeral agent recreated for every pipeline —

starts from nothing. The 5 minutes of `dependency:go-offline` get paid all over again. Two mechanisms: (1) a **cache mount** (`RUN --mount=type=cache,target=/root/.m2`), which keeps the Maven repository on the agent between builds, even when `pom.xml` changes; (2) a **remote cache** — `--cache-from` / `--cache-to` pointing at the registry — which lets a fresh agent fetch the layers of a previous build. With rootless Podman, the cache (layers and *cache mounts*) sits in `~/.local/share/containers/storage` of the user who builds: an agent that runs each job under a different user or `home`, or that wipes its `home`, never has a cache.

Why. "Nothing changed" is true for the sources and false for the cache: the cache is local state on the machine, not a property of the Dockerfile.

Nuance. Ephemeral agents are a deliberate choice (isolation, reproducibility); the fix is to make the cache **explicit and external**, not to keep agents alive longer. And a `--no-cache` added to the pipeline "just to be safe" produces exactly this symptom — permanently.

Example.

```
podman build --cache-to registry.internal/myapp/api-cache --cache-from
registry.internal/myapp/api-cache -t api:1.5 .
podman info --format '{{.Store.GraphRoot}}'    # where this user's cache lives
```

Question 9 — What you lose with distroless

Answer. (1) `podman exec -it ... sh`: no shell means no interactive exploration, no `cat` on a configuration file, no `curl localhost:8080/actuator`. Teams compensate with exposed observability endpoints (`/actuator/health`, `/info`, `/env`), complete structured logs on `stdout`, and `podman cp` to pull a file out. (2) **Diagnostic tools** (`jcmd`, `jstack`, `ps`, `netstat`): nothing to take a *thread dump* with or to list sockets. Teams compensate with a debug *sidecar* container that shares the namespaces (`podman run --pid=container:api --network=container:api debug-image`), or with JMX/Actuator tooling (`/actuator/threaddump`) exposed on the internal network.

Why. Every tool an operator uses to get into a container serves an attacker just as well. Distroless removes both at once, so observability has to move **outside** the image.

Nuance. Distroless images come in a `:debug` variant with a busybox shell — useful in staging, forbidden in production. And Kubernetes offers `kubectrl debug` with ephemeral containers for exactly this need.

Example.

```
podman exec d sh -c ls    # executable file `sh` not found
curl -s localhost:18082/actuator/health    # observability goes through HTTP
```

Question 10 — The cache mount, `VOLUME`, and `# syntax=`

Answer. The data lives in a **cache directory managed by the build engine** (BuildKit or Buildah) on the machine that builds — with rootless Podman, in your user storage. It is mounted into the build container **only while the instruction runs**, then unmounted: nothing is written to a layer, so nothing ends up in the image. A `VOLUME` is the opposite: a declaration stored in the image that creates a volume for the container at **run time**, and does nothing during the build. Without `# syntax=docker/dockerfile:1`: recent Docker versions (BuildKit by default) run `--mount` fine with the current stable syntax — the line only existed to force a newer frontend version. Podman **ignores** the line altogether (Buildah has no frontend), and `--mount` works natively.

Why. The cache mount is a build-engine mechanism; `VOLUME` is a runtime mechanism. The word "mount" is all they share.

Nuance. The cache mount is not shared between machines or users, and its content is never invalidated: a corrupted Maven repository stays in it. `podman system prune --build-cache ...` does not exist yet: you delete the storage, or switch caches with `id=`.

Example.

```
podman build --no-cache -f Dockerfile.cache -t c . 2>&1 | grep dep-      # markers accumulate
from one build to the next
podman run --rm c ls /root/.m2                                          # absent from the image
```

Question 11 — "Multi-stage is useless for Angular"

Answer. Without multi-stage, the final image is the one `ng build` ran in: `node:22-alpine` (~170 MB) **plus** `node_modules` (500 MB to 1 GB) **plus** the TypeScript sources **plus** `dist/` — and you still have to add a server to serve `dist/`. With multi-stage: `nginx:alpine` (64 MB) plus a few MB of static files. That is a factor of 10 to 20, and the content changes in kind: no Node, no sources, no build dependencies.

Why. The static result is precisely the argument **for** multi-stage: since running the app needs nothing that built it, there is no reason to keep any of it.

Nuance. Without a container build, a team can also run `ng build` in CI and copy `dist/` into an `nginx` image in a single step (`COPY dist/ /usr/share/nginx/html`). That is a "multi-stage" whose first stage is the CI itself — valid, but the build is no longer reproducible from the Dockerfile alone.

Example.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'  # 64.2 MB
against 167 MB
```

Question 12 — `/app/dist: no such file or directory`

Answer. The `build` stage sets `WORKDIR /src`, so the build produces `/src/dist`, not `/app/dist`. Fix: `COPY --from=build /src/dist/<project>/browser /usr/share/nginx/html` (the sub-folder depends on the Angular version and the project name). To diagnose instead of guessing: `podman build --target build -t dbg .`, then `podman run --rm dbg find / -name index.html -path '*dist*'.`

Why. `COPY --from` copies from the stage's file system, using **absolute** paths within that stage. A wrong `WORKDIR` or an unexpected `dist/` layout stays invisible until you look inside.

Nuance. Since Angular 17, the default output is `dist/<project>/browser/`; before that, `dist/<project>/`. `--target` saves you from relying on memory.

Example.

```
podman build --target build -t dbg . && podman run --rm dbg ls -R /src/dist | head
```

Question 13 — `RUN mvn test` in the Dockerfile

Answer. In the build stage, **after** compilation and **before** `package` (or as a single `mvn package` without `-DskipTests`): a failing test fails the `RUN`, the build stops, and no image is produced. The drawback: the tests run in an isolated build container — CI gets no usable JUnit report (it sits in a discarded stage unless you extract it with `--target` or `--output`), a test database is hard to reach (Testcontainers needs an engine), and the image build time now includes the tests, even when all you wanted was a rebuild.

Why. The Dockerfile makes a good gatekeeper ("no image without green tests") but a poor reporting tool.

Nuance. The common compromise: CI runs the tests **and** the image build as two jobs, with the build gated on the tests passing; the Dockerfile keeps `-DskipTests` to stay fast. You get the report and the guarantee, at the price of depending on CI.

Example.

```
RUN mvn -q test           # red -> the build stops here
RUN mvn -q package -DskipTests
```

Question 14 — One 250 MB layer or five layers of 280 MB

Answer. The **280 MB image in five layers** deploys faster on a code update: only the volatile layer (a few MB) is transferred; the other four already sit on the nodes and in the registry. The 250 MB single-layer image resends 250 MB with every version. The answer reverses when the nodes have **nothing** yet (first deployment, fresh node, emptied registry, or a tagging strategy that changes everything every time): then $250 < 280$ and the single layer wins — barely.

Why. The transfer cost is the cost of the missing layers, not of the image. Layer stability matters more than layer count.

Nuance. `--squash` (Buildah) or a smaller base can bring the 280 MB down to 250 without giving up the layers: the two criteria are not mutually exclusive. And the gain only exists if the stable layers are **bit-for-bit identical** from one build to the next — which requires reproducible builds (no unpinned `apt-get update` in a "stable" layer).

Example.

```
podman push registry.internal/myapp/api:1.5.1    # stable blobs: instantaneous; only the code
layer is copied
```